

# Hyze Installation Guide

## Getting Started

 Welcome to the installation guide. This comprehensive document will walk you through the setup process step by step. Please ensure you have all prerequisites ready before beginning.

## Essential Prerequisites

Before starting the installation, ensure you have access to the following components:

### Authentication Credentials

-  Server Access Credentials
-  Google Cloud Console Login
-  Domain Provider Login
-  Cloudflare Account Access
-  MongoDB Atlas Credentials
-  Email Service Credentials
-  SMS Gateway Credentials
-  Payment Gateway Access
-  Terms of Service & Privacy Policy

## Required Tools

-  Code Editor (e.g., VS Code – recommended)
-  File Transfer Tool (FileZilla / Terminus)
-  Server Connection Tool (Terminus / PuTTY)
-  Database Manager (Robo 3T / MongoDB Compass)

 **Pro Tip:** Before you begin installation, create a secure document to store and organise all your login credentials and access information.

## Quick Navigation

-  [Server Setup](#)
-  [Database Setup](#)
-  [Cloudflare Setup](#)
-  [Google Cloud Console Setup](#)
-  [Firebase Setup](#)
-  [Email Notifications](#)
-  [Twilio Setup](#)
-  [Payment Gateway Setup](#)
-  [Installation & Deployment](#)



# Server Setup

## ♦ [AWS Setup](#)

-  [Launch EC2 Instance](#)
-  [Allocate & Associate Elastic IP](#)
-  [Configure Security Group](#)
-  [Create S3 Bucket](#)

## ♦ [Vultr Setup](#)

-  [Deploy Server](#)
-  [Set Up Firewall & IP Whitelisting](#)
-  [Configure SSH Access](#)

## ♦ [DigitalOcean Setup](#)

-  [Create Droplet](#)
-  [Assign Floating IP](#)
-  [Configure Firewall & Access Rules](#)



# [Database Setup](#)

-  [Create MongoDB Cluster](#)
-  [Add Database Users & Assign Roles](#)
-  [Whitelist Access IPs](#)
-  [Retrieve Database Connection String](#)



# [Cloudflare Setup](#)

-  [Register or Transfer Domain](#)
-  [Manage DNS Records](#)
-  [Enable Full \(Strict\) SSL/TLS](#)
-  [Generate & Apply SSL Certificate](#)

## Google Cloud Console Setup

-  [Verify Google Account & Billing](#)
-  [Create a New Project](#)
-  [Enable Required APIs](#)
-  [Generate OAuth2 / API Keys](#)

## Firebase Setup

-  [Create Firebase Project](#)
-  [Register Web Application](#)
-  [Create Realtime Database](#)
-  [Generate Admin SDK Key](#)

## Twilio Setup

-  Set Up Twilio Account and Project
-  Generate API Keys
-  Configure Webhooks

## Email Notification Setup

-  Choose Email Service Provider (*e.g., SendGrid, Mailgun*)
-  Configure SMTP Settings
-  Verify Domain and Authentication
-  Integrate with Application



## Payment Gateway Setup

-  Choose Payment Provider (*e.g., Stripe, Razorpay*)
-  Generate API Keys & Secrets
-  Configure Webhooks
-  Integrate Payment Module



## Installation & Setup

1.  [Download the Installation Script \(.sh\)](#)  
Retrieve the installation script to begin the setup process.
2.  Run the Installation Script  
Execute the script to automatically install all required dependencies.
3.  Apply SSL Certificate  
Add your SSL certificate to the server configuration to enable secure HTTPS access.
4.  Clone the Project Repository  
Clone the source code repository and rename the project directory if needed.
5.  Update URLs & Credentials  
Edit the configuration files with your custom domain, API keys, and access credentials.
6.  Build the Project  
Compile the project to generate production-ready files.
7.  Deploy to Server & Start Services  
Upload the build to your server and launch all necessary backend/frontend services.
8.  Run Database Migration  
Use the provided migration APIs to initialize essential data structures in the database.

9.  Configure Admin Panel Settings

Log into the admin panel and input all API keys, tokens, and configuration parameters.

# Amazon AWS Setup

## 1. Login or Sign Up to AWS Console

 [Open AWS Management Console](#)

---

## EC2 Instance Creation

 **Need a Visual Guide ?**

 [Click Here for Picture Guide \(EC2\)](#)

## Step-by-Step EC2 Instance Launch

### Step 1: Log In or Create an Account

- Visit: <https://aws.amazon.com/console>
- Use your credentials to sign in, or register a new account.

### Step 2: Choose Your AWS Region

- In the top-right, open the Region dropdown.
- Select the region closest to your end users or clients.

### Step 3: Locate EC2

- Use the search bar and type **EC2**.
- Select EC2 from the results.

### Step 4: Launch EC2 Instance

1. Click **Instances** in the EC2 dashboard.
2. Click **Launch instances**.

### Step 5: Name Your Instance

- Under the “Name” field, enter a name like `MyProjectServer`.

### Step 6: Choose an Amazon Machine Image (AMI)

Select:

- Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
- AMI ID (x86): `ami-0e35ddab05955cf57`
- AMI ID (Arm): `ami-0429d68a1cd41ca80`

### Step 7: Select Instance Type

- Choose: `c5.large`

### Step 8: Create SSH Key Pair

1. Click Create new key pair
2. Name it (e.g., `myproject-key`)
3. Format: `.pem`
4. Click Create key pair (downloads the file)

 Keep the `.pem` file safe — it's required for SSH access.

### Step 9: Configure Security Group

1. Select Create security group
2. Add inbound rules:
  - SSH (22)
  - HTTP (80)
  - HTTPS (443)



3. Add other ports if needed.

## Step 10: Configure Storage

- Set root volume to at least 30 GB

## Step 11: Launch Instance

- Click Review and Launch
- Confirm settings and launch the instance.

## Step 12: Get Public IP

- Go to your EC2 instance in the dashboard
- Copy the Public IPv4 Address

# Elastic IP Allocation & Association

## **Need a Visual Guide?**

 [Click Here for Picture Guide \(Elastic IP\)](#)

## Steps to Allocate and Attach Elastic IP

### Step 1: Open AWS Console

 [AWS Console](#)

### Step 2: Search for EC2 and Open

- Use the search bar and select EC2

### Step 3: Navigate to Elastic IPs

- In the EC2 sidebar, click Elastic IPs

### Step 4: Allocate New Elastic IP

1. Click Allocate Elastic IP address
2. Choose Amazon pool
3. Click Allocate

### Step 5: Associate Elastic IP

1. Select the new IP address
2. Click Actions → Associate Elastic IP address
3. Choose your EC2 instance
4. Click Associate

# Configure Security Group

 A Security Group acts as a virtual firewall to control inbound and outbound traffic for your EC2 instance.

---

## Step-by-Step Guide

### Step 1: Select "Create Security Group"

While launching your EC2 instance:

- Navigate to the "Configure Security Group" step.
  - Select "Create a new security group" (unless you already have one configured).
- 

### Step 2: Add Inbound Rules

Configure rules to allow traffic to your server:

| Type  | Protocol | Port Range | Source           | Purpose                  |
|-------|----------|------------|------------------|--------------------------|
| SSH   | TCP      | 22         | My IP / Anywhere | Secure shell access      |
| HTTP  | TCP      | 80         | Anywhere         | Allow web traffic        |
| HTTPS | TCP      | 443        | Anywhere         | Secure web traffic (SSL) |

 You can add additional ports based on your application's requirements (e.g., 3000, 5000, etc.).

---

### Optional: Customize Rules

- Restrict SSH (Port 22) to **your IP** only for better security.
  - You can **add custom TCP/UDP ports** as needed for backend/frontend apps.
- 

### Step 3: Save and Continue

- After configuring rules, scroll down and click **“Review and Launch”**.
- Your security group will be applied to this EC2 instance.



# AWS S3 Bucket Setup



Need a Visual Guide?



[Click Here for Picture Guide \(S3\)](#)

---



## PART 1: Create S3 Bucket & Upload Files



### Step 1: Open S3 Dashboard



[Open S3 Console](#)



### Step 2: Create New Bucket

1. Search for S3 Buckets
2. Click Create Bucket
3. Name it (e.g., `myproject-bucket`)
4. Match region with your EC2 region



### Step 3: Set Permissions

- Enable ACLs
- Set Bucket Owner Preferred
- Allow public access (if required)



### Step 4: Upload Files

1. Open your bucket
2. Click Upload

3. Add files or folders and configure permissions

✅ Files can be accessed publicly (if public access is enabled)

---

## 🔑 PART 2: Create AWS Access Key

💡 Need a Visual Guide?

👉 [Click Here for Picture Guide \(Access Key\)](#)

---

### ✅ Step 1: Navigate to Security Credentials

1. Click your account name
2. Select Security Credentials
3. Go to the Access Keys section

### 🔑 Step 2: Generate Access Key

1. Click Create Access Key
2. Acknowledge the warning and proceed
3. Click Download .csv

⚠️ Store this file securely — it contains your Access Key ID and Secret Access Key



# MongoDB Atlas Setup

🚀 Step-by-Step Guide to Set Up a Free MongoDB Cluster on Atlas



Need a Visual Guide?



[Click Here for Picture Guide \(MongoDB\)](#)

---



## 1. Log In or Sign Up

- Visit: <https://www.mongodb.com/cloud/atlas>
  - Click “Sign In” (top-right), or “Start Free” if you're new.
  - Sign in using Google, GitHub, or Email/Password.
- 



## 2. Create a Cluster (Free Tier – M0)

- Click “Build a Database”
  - Choose Shared Cluster (M0 Free Tier)
  - Click “Create”
- 



## 3. Select Cloud Provider & Region

- Choose your preferred Cloud Provider (e.g., AWS, Google Cloud)
- Select the Region closest to your users (e.g., AWS / N. Virginia)
- You may rename the cluster, or leave it as default
- Click “Create Cluster”



*Deployment takes approximately 1-2 minutes*

---

## 4. Create a Database User

- Go to Database → Database Access
- Click “Add New Database User”

Fill in the details:

- Username: (e.g., `myUser`)
  - Password: Set manually or auto-generate
  - Privileges: Leave as default — *Read and Write to Any Database*
  - Click “Add User”
- 

## 5. Configure Network Access (IP Whitelisting)

- Go to Network Access
- Click “Add IP Address”

Choose one:

- Add your current IP
  - Or use `0.0.0.0/0` to allow all IPs (*not recommended for production*)
  - Click “Confirm”
- 

## 6. Get the Connection String

- Go to Database → Clusters
- Click “Connect”
- Choose “Connect Your Application”
- Select Node.js and your version



Copy the connection string format:

```
mongodb+srv://<user>:<password>@cluster0.xxxxx.mongodb.net/?retryWrites=true&w=majority
```

 Important:

- Replace <user> and <password> with your actual database password
- Store this string securely — you'll use it in your backend .env file or config

# Cloudflare Setup

💡 Need a Visual Guide?

👉 [Click Here for Picture Guide \(Cloudflare\)](#)

🌐 Full Cloudflare Setup Guide

(Domain, DNS, SSL, Name Servers, and Origin Certificate)

---

## 🔑 Step 1: Log In to Cloudflare

- Visit: <https://dash.cloudflare.com/>
  - Click on “Account Home” in the sidebar
- 

## 🌐 Step 2: Add Your Domain

- Enter your domain name (e.g., [example.com](#))
  - Click “Continue”
- 

## 📦 Step 3: Select a Plan

- Choose a plan
    - ✅ *The Free Plan is sufficient for most projects*
  - Click “Select Plan”
- 

## 🔧 Step 4: Add DNS Records

For each subdomain you want to use (e.g., [panel.example.com](#), [api.example.com](#)):

1. Click “Add Record”

2. Use the following settings:
    - **Type:** A
    - **Name:** your subdomain (e.g., **panel**)
    - **IPv4 Address:** your server's public IP
  3. Repeat for all required subdomains
- 

## Step 5: Save and Activate DNS

- Click “Save”
  - Then click “Continue” to activation
- 

## Step 6: Confirm DNS Setup

- On the next screen, click “Confirm”
- Cloudflare will display two Name Servers

 *Copy both name servers — you'll need them in the next step*

---

## Step 7: Update Name Servers at Domain Registrar

- Log in to your domain provider (e.g., GoDaddy, Namecheap)
- Navigate to the DNS or Name Server settings
- Replace the existing name servers with the **two** from Cloudflare
- Click **Save Changes**

 *DNS changes can take up to 24 hours to propagate globally*

---

## ✅ Step 8: Finalize DNS Setup in Cloudflare

- Return to Cloudflare
  - Click “Continue” to complete the setup
- 

## 🛡️ Step 9: Enable Full (Strict) SSL

1. Go to SSL/TLS → Overview
  2. Set SSL Mode to Full (Strict)
  3. Click Confirm
- 

## 📄 Step 10: Create Cloudflare Origin Certificate

1. Navigate to SSL/TLS → Origin Server
2. Click “Create Certificate”
3. Cloudflare will generate two files:
  - `origin-cert.pem` (Public Certificate)
  - `private-key.pem` (Private Key)

✅ *Download and store both files securely*

---

## 📁 Step 11: Upload Origin Certificate to Your Server

Use SSH or your hosting control panel to:

1. Upload `origin-cert.pem` and `private-key.pem` to your server
2. Configure your web server (e.g., NGINX or Apache) to use these files for SSL

✅ Done! Your domain is now protected with Cloudflare’s DNS and Full SSL setup.

# Google Cloud Console Setup

## 1.4 Credentials & API Access

Use these steps to integrate Google Maps, configure billing, and manage Google Login.

---

### 1.4.1 Project & Billing

#### Step 1: Verify Billing Setup

- Visit: <https://console.cloud.google.com/billing>
- Confirm that billing is enabled for your project

#### Step 2: Use or Create a Project

- Use an existing project
- Or create a new one if needed

#### Step 3: Get Console Access

- Request access from the client if you don't have it
- 

### 1.4.2 Google Maps API Key Setup

#### Step 1: Enable Required APIs

Go to the Google API Library, and enable:

- Geocoding API
- Geolocation API
- Places API

- Routes API
- Maps Static API
- Maps JavaScript API

#### ✅ Step 2: Generate API Key

- Go to APIs & Services → Credentials
- Click Create Credentials → API Key

#### ✅ Step 3: Rename the Key

- Name the API Key: [Web Manual](#)

#### ✅ Step 4: Restrict API Key (Web)

- Under Key Restrictions, choose:  
“HTTP Referrers (websites)”
- Add domain:  
[\\*.example.com/\\*](#) *(or any domain you want to allow)*

#### ✅ Step 5: Restrict API Key (Android)

- Select Android apps
- Add:
  - Package name
  - SHA-1 fingerprint

#### ✅ Step 6: Restrict API Key (iOS)

- Select iOS apps
  - Add:
    - iOS Bundle ID
-

## Google Login OAuth Setup

### ✓ Step 1: Go to Google Cloud Console

- Visit: <https://console.cloud.google.com/>
- Log in with your Google account

### ✓ Step 2: Create a New Project

- Click the project dropdown → New Project
- Name your project → Click Create

### ✓ Step 3: Enable Google Identity API

- Go to APIs & Services → Library
- Search: Google Identity Services
- Click → Enable

### ✓ Step 4: Configure OAuth Consent Screen

- Go to: APIs & Services → OAuth consent screen
- Choose External
- Fill out:
  - App name
  - Support email
  - Developer contact
- *(Add test users if needed)*

### ✓ Step 5: Create OAuth Client ID

- Go to Credentials → Create Credentials → OAuth client ID

- Choose Web Application
  - Add:
    - Authorized JavaScript Origins: <http://localhost:3000>
    - Authorized Redirect URIs:  
<http://localhost:3000/auth/callback>
  - Click Create
  - Save your:
    - Client ID
    - Client Secret
- 

## OAuth Integration

### ✓ Step 6: Google OAuth Login URL

Redirect users to the following URL:

[https://accounts.google.com/o/oauth2/v2/auth?  
client\\_id=YOUR\\_CLIENT\\_ID  
&redirect\\_uri=YOUR\\_REDIRECT\\_URI  
&response\\_type=code  
&scope=email profile openid](https://accounts.google.com/o/oauth2/v2/auth?client_id=YOUR_CLIENT_ID&redirect_uri=YOUR_REDIRECT_URI&response_type=code&scope=email profile openid)

### ✓ Step 7: Exchange Code for Auth Token

Make a **POST** request to:

<https://oauth2.googleapis.com/token>

With this request body:

client\_id=YOUR\_CLIENT\_ID  
client\_secret=YOUR\_CLIENT\_SECRET  
code=AUTH\_CODE  
grant\_type=authorization\_code  
redirect\_uri=YOUR\_REDIRECT\_URI



## ✓ Step 8: Use the Token

Example JSON response:

```
{  
  "access_token": "ya29...",  
  "expires_in": 3599,  
  "refresh_token": "1//...",  
  "scope": "openid email profile",  
  "token_type": "Bearer",  
  "id_token": "eyJhbGciOi..."  
}
```

- **access\_token**: Use it to fetch user info
- **id\_token**: Decode it to get user profile (email, name, etc.)

# Firebase Setup

## Step-by-Step Guide to Set Up Firebase

 *Need a Visual Guide?*

 [Click Here for Picture Guide \(Firebase\)](#)

---

### ◆ Firebase Console Setup

1. Visit: <https://console.firebase.google.com/>
  2. Click "Get started with a Firebase project"
  3. Enter a Project Name
  4. Accept all terms and conditions
  5. Enable Google Analytics:
    - Choose your country (e.g., *India*)
    - Accept the Analytics terms
  6. Click Create Project
- 

### ◆ Web App Registration

1. After project creation, click the Web (</>) icon to register a new web app
  2. Enter a nickname (e.g., *Web App*)
  3. Click "Register App"
  4. Click "Continue to the Console"
-

## ◆ Realtime Database Setup

1. Go to Build > Realtime Database from the left sidebar
2. Select a region closest to your users
3. Click “Create Database”
4. Click “Next” → then “Enable”
5. Navigate to the Rules tab and change the default rules:

### ✗ Default Rules:

```
{
  "rules": {
    ".read": false,
    ".write": false
  }
}
```

### ✓ Updated Rules (for development use only):

```
{
  "rules": {
    ".read": true,
    ".write": true
  }
}
```

🛡️ Note: This configuration is only for development. Use stricter rules in production.

---

## ◆ Firebase Admin SDK (Service Account)

1. Go to Project Settings → Service Accounts
2. Click “Generate New Private Key”
3. A `.json` file will download — this is your Service Account credentials

4. Use this file in your **server-side code** (e.g., Node.js) to initialize Firebase Admin SDK

 *Important:*

- Store this file securely
- Do NOT commit it to public repositories



# System Installation & Deployment Guide

1. [Download Installation File Setup](#)
2. [NGINX Configuration](#)
3. [Project Branding & Language Setup](#)
4. [Environment Variable Configuration](#)
5. [Project Build & Deployment](#)
6. [Postman Migration API](#)
7. [Admin Panel Configuration](#)



## [Download Installer File](#)

After downloading, place the file on your server and follow the steps below:

```
# Make the file executable  
chmod +x FilePath/FileName
```

```
# Run the installation script  
./FilePath
```

Choose the installation mode as per your requirements.

The script will install:

- Node.js
- NGINX
- Redis (if applicable)
- PM2
- Other dependencies as needed

# NGINX Configuration

## Step 1: Create NGINX Site Config

```
mkdir /etc/nginx/sites-available/myprojectName
```

Place **origin.pem** and **private.pem** in **/etc/ssl/myprojectName/**

Set file upload limit:

```
client_max_body_size 100M;
```

## Sample NGINX Config

- server {
- listen 80;
- server\_name api.clientdomain.com;
- return 301 https://\$host\$request\_uri;
- }
- 
- server {
- listen 443 ssl;
- server\_name api.clientdomain.com;
- ssl\_certificate /etc/ssl/myprojectName/origin.pem;
- ssl\_certificate\_key /etc/ssl/myprojectName/private.pem;
- 
- location / {
- proxy\_pass http://localhost:3000;
- proxy\_http\_version 1.1;
- proxy\_set\_header Upgrade \$http\_upgrade;
- proxy\_set\_header Connection 'upgrade';
- proxy\_set\_header Host \$host;
- proxy\_cache\_bypass \$http\_upgrade;
- }
-

- location /api/gmaps/maps {
- proxy\_pass https://maps.googleapis.com/maps/;
- proxy\_pass\_request\_body on;
- proxy\_set\_header X-Forwarded-For \$remote\_addr;
- }
- 
- location /api/gmaps/routes {
- proxy\_pass https://routes.googleapis.com/;
- proxy\_pass\_request\_body on;
- proxy\_set\_header X-Forwarded-For \$remote\_addr;
- }
- }

## ✅ Step 2: Test & Restart NGINX

- nginx -t
- sudo service nginx restart

sudo service nginx status

---

## Project Branding & Language Setup

Step 1: Update Project Name Update all placeholders (e.g., **YourProjectName**) in:

- Web app titles
- Firebase project name
- PM2 & NGINX config
- Mobile display names
- Firebase object (if using Firebase messaging)

## Step 2: Set Language & URLs

- Ask for preferred language ([en](#), [hi](#), [fr](#), etc.)
- Add IP URLs in [server.js](#) in all panels
- Update [commonconfig.ts](#) for CORS whitelisting
- Add S3 image URLs if using AWS S3

### Sample CORS Whitelist:

- let whitelist = [
  - "http://YourServerIP",
  - "<https://YourServerIP>"
- ];

### Sample URL Constants:

- IMAGE\_URL: "https://s3.example.com",
- SOCKET\_URL: "<https://socket.example.com>",
- API\_URL: "<https://api.example.com>",
- HISTORY\_URL: "https://history.example.com"

## Step 3: Angular [environment.prod.ts](#)

- export const environment = {
  - IMAGE\_URL: "IMAGE\_URL", // (S3 URL if using S3 Bucket)
  - SOCKET\_URL: "SOCKET\_URL",
  - HISTORY\_URL: "HISTORY\_URL",
  - PREDICTION\_URL: "PREDICTION\_URL", // (Admin Panel only)
  - defaultLang: "Client's Preferred Language",
  - production: true, // (Set false for development)
  - firebaseConfig: {
    - apiKey: "",
    - authDomain: "",
    - databaseURL: "",
    - projectId: "",



- storageBucket: "",
  - messagingSenderId: "",
  - appId: "",
  - measurementId: ""
  - }
- };
- 

## Environment Variable Configuration

### Backend **.env.production**

- MONGODB\_URI=YOUR\_MONGODB\_URI
- LOG\_DB\_URI=YOUR\_LOG\_DB\_URI
- LOCATION\_DB\_URI=YOUR\_LOCATION\_DB\_URI
- CHAT\_DB\_URI=YOUR\_CHAT\_DB\_URI
- HISTORY\_DB\_URI=YOUR\_HISTORY\_DB\_URI
- JWT\_SECRET=YOUR\_SECRET\_TOKEN
- ENCRYPTION\_KEY=YOUR\_SECRET\_KEY
- PORT=4000
- HISTORY\_PORT=4001
- SOCKET\_PORT=4002
- DEFAULT\_LANGUAGE=en
- DEFAULT\_LANGUAGE\_NAME=English
- REDIS\_HOST="127.0.0.1"
- REDIS\_PORT=6379
- REDIS\_PASSWORD=""
- REDIS\_DB=0
- SSL\_CERT\_FILE="../../certificate/file.crt"
- SSL\_KEY\_FILE="../../certificate/file.key"
- REDLOCK\_MAX\_LIFE=30
- APP\_NAME=Your\_app\_name

## Project Build & Deployment

 Step 1: Transfer Code to Server Upload backend and frontend project files.

 Step 2: Initialize Backend

- `cd backend`
- `npm install`

`npm run init-settings`

 Step 3: Start Backend with PM2

`pm2 start ecosystem.config.js --env production`

 Step 4: Start Frontend Panels

- `cd /path/to/PanelFolder`

`pm2 start server.js --name=PanelName`

 Step 5: Save PM2 State

`pm2 save`

`pm2 startup`

---

## Postman Migration API

1. Import Postman Collection.
2. Use API PUT `/api/language/strings/migration_string`

Sample API Request:

- {
- "stringTypes": [],
- "codes": ["LANG\_CODES"],
- "actionType": 1,
- "serverUrl": "https://demoapi.starlor.ai",
- "serverToken": "[ADMIN\_SERVER\_TOKEN]"
- }

*Replace with your Admin Bearer Token and required language codes.*

---

## Admin Panel Configuration

1. Add Map Keys:
  - Admin → Map Settings
2. Add Config Keys:
  - Web URL, Image Storage, Social Login, Payment Methods
3. Add Notification Keys:
  - Email, Push, SMS, Live Push Notifications
4. Add Logos:
  - Admin → Logo Settings
5. Create Business:
  - Based on your use case

6. Start Api Logs :

- In DB -> Conf Settings -> keepApiLogs to True and save settings

7. Add Privacy Urls:

- Add All Terms and Privacy Policy in

Setting > Configuration Setting > Terms & Privacy

## Vultr Setup

 This section is currently under development.

Stay tuned for a complete step-by-step guide on how to:

-  Create and configure Vultr Instances
-  Set up Networking and Firewalls
-  Connect and Deploy Your Project

 **Pro Tip:** Bookmark this section so you can easily return once the full guide is published!

---

## DigitalOcean Setup

 This section is currently under development.

You will soon find detailed instructions for:

-  Launching a DigitalOcean Droplet
-  Securing Your Server
-  Connecting via SSH and Configuring DNS

 **Need Help Sooner?**

Check out [DigitalOcean Docs](#) for official tutorials and setup guidance.

## Twilio Setup

 *This section is currently under development.*  
Stay tuned for a detailed step-by-step guide on how to:

- Set up a Twilio account
  - Generate API credentials
  - Send SMS and voice notifications
-  *Pro Tip:* Bookmark this section for future updates!
- 

## Email Notification Setup

 *This section is currently under development.*  
Soon you'll find a comprehensive guide for:

- SMTP setup (e.g., Gmail, SendGrid, Mailgun)
  - Verifying sender domains
  - Integrating email alerts into your app
-  *Useful for: User verification, system alerts, password resets.*
- 

## Payment Gateway Setup

 *This section is currently under development.*  
We're working on step-by-step instructions for:

- Setting up Stripe, Razorpay, or PayPal
  - Creating and securing API keys
  - Handling payments, webhooks, and refunds
-  *Essential for projects requiring billing or eCommerce.*